

Partie 1 : Élections municipales (10 points) -> copie n°1

bureau: numero: int
↑ inscrits: int
ville: list(dict) votes: dict

Une ville organise des élections municipales. Les résultats sont stockés sous forme d'une liste de résultats de bureaux de vote. Dans la suite les résultats d'une municipalité sont appelés **ville** et un bureau de vote est appelé **bureau**. Chaque bureau est un dictionnaire avec 3 couples dont les clés sont "numero" (numéro du bureau de vote), "inscrits" (nombre d'inscrits sur les listes municipales) et "votes". La structure de "votes" est également un dictionnaire dont les clés sont les noms des candidats et les valeurs le nombre de voix obtenues dans le bureau de vote. A titre d'exemple pour 3 candidats (il peut y en avoir moins ou plus) :

```
ville = [  
    {"numero": 1, "inscrits": 500, "votes": {"Marinier": 120, "Blaque": 95, "Foret": 60}},  
    {"numero": 8, "inscrits": 400, "votes": {"Foret": 70, "Marinier": 80, "Blaque": 110}},  
    {"numero": 15, "inscrits": 760, "votes": {"Blaque": 130, "Marinier": 150, "Foret": 90}},  
    {"numero": 4, "inscrits": 550, "votes": {"Blaque": 85, "Foret": 100, "Marinier": 90}},  
    {"numero": 9, "inscrits": 380, "votes": {"Marinier": 70, "Blaque": 60, "Foret": 80}}  
]
```

Remarque : les paramètres des fonctions sont parfois volontairement omis : à vous de les définir.

- 1.1 Écrire la fonction `ajoute_bureau(bureau, ville)` qui ajoute les résultats d'un bureau de vote aux résultats de la ville. Si les résultats du bureau ont déjà été enregistrés, on affichera un message : "Résultats du bureau déjà saisis".
- 1.2 A l'aide de la fonction précédente, écrire la fonction `ajoute_bureaux()` qui ajoute les résultats d'un ou plusieurs bureaux de vote aux résultats de la ville.
- 1.3 Écrire la fonction `taux_participation()` qui retourne le taux de participation (en pourcentage) d'un bureau de vote.
- 1.4 A l'aide de la fonction précédente, écrire la fonction `meilleur_taux()` qui, pour une ville donnée, retourne le **numéro du bureau** qui a le meilleur taux de participation.
- 1.5 Écrire la fonction `total_candidats()` qui, pour une ville donnée, calcule et retourne un dictionnaire contenant le nombre de voix total obtenues (valeur) par candidat (clé).

Exemple de retour de la fonction : {'Marinier': 510, 'Blaque': 480, 'Foret': 400}

- 1.6 Écrire une fonction `candidats_second_tour(ville, seuil)` qui retourne une liste avec les candidats qui ont obtenu un pourcentage supérieur au seuil passé en paramètre.

Exemple: `candidats_second_tour(ville, 32.0)` retourne ['Marinier', 'Blaque'] car les scores des candidats sont : 36.69% pour Marinier, 34.54% pour Blaque, et 28.25% pour Foret.

- 1.7 Écrire un décorateur `detection_fraude` qui vérifie que le premier paramètre de la fonction décorée correspond à un bureau avec des résultats cohérents c'est à dire que le nombre de votes exprimés est inférieur ou égal au nombre d'inscrits. Si ce n'est pas le cas, on affiche un message de fraude sur le numéro de bureau de vote et la fonction retourne 0.

- 1.8 Comment applique-t-on le décorateur et sur quelle(s) fonction(s) (des questions 1 à 6) va-t-on l'appliquer ?

Partie 2 : programmation objet pour l'internet des objets (IoT) (10 points) -> copie n°2

La plateforme SmartLink permet de superviser des équipements IoT (IoT pour Internet of Things). Une plateforme est composée de plusieurs équipements IoT (capteurs, actionneurs, nœuds, ...) communicants via un ou plusieurs protocoles de communication. L'objectif de cet exercice est de développer une partie de cette application.

Un **objet de la classe EquipementIoT** représente un équipement générique de la plateforme et possède les attributs suivants : **_id** (entier positif), **_nom** (chaîne de 30 caractères max) et **_protocoles** (liste de protocoles acceptés par l'équipement, chaque protocole est désigné par un nom)

- 2.1 Définir la classe `EquipementIoT` avec un constructeur permettant d'initialiser ses attributs à l'aide des informations transmises en paramètres, ainsi que des décorateurs `property` permettant d'accéder / modifier les attributs `_id` et `_nom`. L'attribut `_protocoles` est vide à l'initialisation et sera complétée grâce à la méthode `ajouter_protocole()` qui ajoute un ou plusieurs protocoles passés en paramètres. `print(e)` provoque l'affichage des attributs `_id` et `_nom` de l'objet `e` de la classe `EquipementIoT`.

Les sous-classes de `EquipementIoT` sont les suivantes :

- `CapteurIoT` : permet d'effectuer une mesure.
- `ActionneurIoT` (**non demandée dans cet exercice**) : effectue une action selon son état OFF ou ON. En plus des attributs hérités, un actionneur possède un attribut `_etat` dont la valeur est "OFF" ou "ON". L'appel de la méthode `collecte()` sur un objet `ActionneurIoT` retourne son état.
- `NoeudIoT` : regroupe plusieurs capteurs, actionneurs ou autres nœuds.

2.2 Classe `CapteurIoT` :

En plus des attributs hérités, un objet `CapteurIoT` possède un attribut `_mesure` dont la valeur initialisée à `None` sera modifiée par l'appel de la méthode `collecte()`. La méthode `collecte()` affecte une valeur aléatoire à l'attribut `_mesure` et retourne cette même valeur. Vous pouvez utiliser la fonction `randint(min, max)` du module `random` qui retourne un entier aléatoire compris entre `min` et `max`.

`print(c)` provoque l'affichage des attributs `_id` et `_nom` de l'objet `c` de la classe `CapteurIoT`.

Définir la classe `CapteurIoT` selon la description ci-dessus.

2.3 Classe `NoeudIoT` :

En plus des attributs hérités, un objet `NoeudIoT` possède un attribut `_equipements` qui liste les équipements IoT regroupés par le nœud. Cet attribut est initialisé à la liste vide. La liste sera complétée avec `e` par la méthode `connecter(e)`. La méthode `connecter(e)` ajoute l'objet `e` dans la liste des équipements regroupés par le nœud à condition que `e` soit un équipement IoT (`CapteurIoT`, `ActionneurIoT` ou `NoeudIoT`) et qu'il ait au moins un protocole en commun avec le nœud. Un message affichera si la connexion a pu se faire ou non.

La méthode `collecte()` construit et retourne un dictionnaire {clé : valeur} dont les clés sont les id des équipements regroupés dans le nœud et la valeur est :

- soit la mesure si c'est un capteur,
- soit l'état si c'est un actionneur,
- soit le dictionnaire du regroupement si c'est un nœud.

`print(n)` provoque l'affichage des attributs `_id` et `_nom` de l'objet `n` de la classe `NoeudIoT`.

Définir la classe `NoeudIoT` selon la description ci-dessus.

2.4 On voudrait pouvoir fusionner 2 objets `NoeudIoT` `n1` et `n2` avec l'opérateur `+`. Le résultat est un nouvel objet `NoeudIoT` dont l'id est la somme des id de `n1` et `n2`, le nom est la concaténation des noms de `n1` et `n2`, les protocoles sont l'union des protocoles des 2 nœuds et les équipements sont l'union des équipements des 2 nœuds. Attention à ne pas avoir de doublons dans les deux listes.

Modifier la classe `NoeudIoT` en conséquence.

2.5. Soit l'extrait de `main()` suivant :

```
def main() :  
    protocoles = ["ble", "zigbee", "wifi", "uwb", "loran"]  
    thermometre = Capteur(25, "Température")  
    luminosite = Capteur(2, "Luminosité")  
    ventilo = Actionneur(47, "Ventilateur")  
    led = Actionneur(10, "Lumière")  
    noeud_a = NoeudIoT(6, "routeur_hall")  
    ...
```

Continuer le code qui :

- crée un objet `noeud_b` dont le nom est "routeur_salle" et l'id est 7,
- lui ajoute les protocoles "ble", "zigbee" et "wifi",
- connecte à `noeud_b` les équipements compatibles parmi `thermometre`, `ventilo`, `led` et `luminosite`,
- crée un nouveau nœud `noeud_lab` qui correspond à la fusion de `noeud_a` et `noeud_b`,
- modifie son nom pour l'appeler "routeur_labo",
- affiche l'id et le nom de `noeud_lab` ainsi que les données de collecte de `noeud_lab`.
- Affiche l'erreur si une exception est déclenchée.